

Compte Rendu de TP 2 : Sécurité

Emin Belkheir - Laure Bantsoukissa

Mars 2026

Partie 3 : AES (Analyse mémoire)

Dans cette partie, on a cherché à récupérer une clé de chiffrement directement depuis la RAM. On a lancé un chiffrement avec `openssl` en utilisant l'algorithme `aes-256-ecb` avec l'option `-nosalt`.

Pour analyser la mémoire, on a suspendu le processus avec `ctrl+z` et on a utilisé `gcore` de `gdb` afin de créer une image complète de la RAM à l'instant souhaité sans arrêter définitivement le programme.

```
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from openssl...
(No debugging symbols found in openssl)
(gdb) run
Starting program: /usr/bin/openssl aes-256-ecb -nosalt -out testAES.enc -p
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter AES-256-ECB encryption password:
Verifying - enter AES-256-ECB encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
Key=1B7BC6D413F51DB8B44A540EC04B1F3DBF2B480CCE1386BAD7F672FB09FF96D
^Z
Program received signal SIGTSTP, Stopped (user).
internal_syscall_cancel (a1=0, a2=93824903692144, a3=8192, a4=a4@entry=0, a5=a5@entry=0,
a6=a6@entry=0, nr=0) at ./nptl/cancellation.c:44
warning: 44 ./nptl/cancellation.c: Aucun fichier ou dossier de ce nom
(gdb) gcore
Saved corefile core.13117
(gdb) Quit
(gdb) Quit
```

FIGURE 1 – Création d'une image mémoire du processus openssl avec `gcore`

Une fois le fichier `core` récupéré, on a utilisé `aeskeyfind` avec l'option `-t` pour scanner le dump et retrouver la clé AES en fonction d'un seuil d'erreur. Le programme a réussi à isoler la clé à 100%.

On a aussi vu comment générer une clé de 32 octets proprement à partir de `/dev/urandom` avec `xxd` pour avoir une source d'entropie fiable.

```

1b7bc6d413f51dbbb44a540ec04b1f3dbf2b480ccce1386bad7f672fb09ff96d
Keyfind progress: 100%
blm4897a@u6-104-05:~/secu/aeskeyfind-master$ ./aeskeyfind -t 50 ../CryKeX-master/core.11145
1b7bc6d413f51dbbb44a540ec04b1f3dbf2b480ccce1386bad7f672fb09ff96d
Keyfind progress: 100%
blm4897a@u6-104-05:~/secu/aeskeyfind-master$ ^C
blm4897a@u6-104-05:~/secu/aeskeyfind-master$ gdb --args
gdb: '--args' specified but no program specified
blm4897a@u6-104-05:~/secu/aeskeyfind-master$ gdb --args openssl aes-256-ecb -nosalt -out testAES.enc
-p
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".

blm4897a@u6-104-05:~/secu/CryKeX-master$ xxd -l 32 -p /dev/urandom > clef-aes-256

```

FIGURE 2 – Génération de la clé AES avec /dev/urandom

Partie 3 : RSA (Chiffrement asymétrique)

L'objectif ici était de comprendre comment échanger un fichier de manière sécurisée entre membres du groupe.

Méthode pour chiffrer à destination d'un autre membre :

On génère d'abord une clé que seul le destinataire pourra utiliser pour déchiffrer.

1. On génère la clé privée : `openssl genrsa -out cle_privee_B.pem 2048`
2. On extrait la clé publique : `openssl rsa -in cle_privee_B.pem -pubout -out public_B.pem`
3. On chiffre/déchiffre le fichier avec la sous-commande `pkeyutl` d'openssl :
 - Chiffrer : `openssl pkeyutl -encrypt -pubin -inkey public_B.pem -in f.txt -out f.enc`
 - Déchiffrer : `openssl pkeyutl -decrypt -inkey cle_privee_B.pem -in f.enc -out f_decrypted.txt`

On a aussi manipulé des clés privées protégées par mot de passe (`-aes256`), ce qui génère un bloc ENCRYPTED PRIVATE KEY

```
blm4897a@u6-104-05:~/secu/rsa-keyfind-master$ openssl genrsa -out clef.pem -aes256
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
blm4897a@u6-104-05:~/secu/rsa-keyfind-master$ ls
clef.pem LICENSE Makefile README rsa-keyfind.cpp rsa-keyfind.o
blm4897a@u6-104-05:~/secu/rsa-keyfind-master$

celin@LAPTOP-6H04REEK:/home/secu/rsa-keyfind$ openssl rsa -in private_key.pem
-outform PEM -pubout -out public_key.pem
writing RSA key
celin@LAPTOP-6H04REEK:/home/secu/rsa-keyfind$ cat p_key.pem
private_key.pem public_key.pem
celin@LAPTOP-6H04REEK:/home/secu/rsa-keyfind$ cat public_key.pem
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAvuPI5wwRMttE0+gqYeDp
BrB8EKitI98LHshGrzarj0Zc8ehYqt0tW7BMDcq6/NZ+aHLj6AXLRn52H8FHON6S
C02fLaLmd2ySgwKsYogbURYNr/i2ySYQ0XHXEzzD18aLBcUezIhiFGG+49zxoMvP
JhPGLsbv7bDKl0JikwxDsgwn/tvFdEegDjIVppJlt09EEH7cKObNUZYJf0+0/40q
4bWdldozGhMZw+4DxmFUQq6Tudpszc01nBB/ejKxKNZLnALU/d0gdTsdLJdQz9g
0q0umd/vULgNOHgirt0LiireuJ9zy0zfMwgHtCwW20jDk31bK75YtoVC6xpr7Nle
gwIDAQAB
-----END PUBLIC KEY-----
```

FIGURE 3 – Génération de la clé privée

```
5/TWZ3i6e3yF67Kr+0MyLY6xX1phAdX3F2NV2HowDsgJjzp0seqw2nfFwbVd6Y0S
GrQLBX/15d0+JP7JybzJlB4U2reFXLqhsIJ622T10702uIjytc8r6uYXfbxYk
Ys2Xk4iLmXj0ioFA1ak2HxKJ7W0c00R1aVtMiI9kGcdRCfDZIKVR17gKh7mtzxI
sa0RPaoVPEKwu80qRFDUK2CDBcN1CMZ1cxVgMy298YsrVVLX/BJoBbwZ63gzzX
A61bPkfy8Hj2n0ceIqKfbb2fwsH4Pk2fr8300wtncLYJSeWlfb/Y5Nv+P8y4N+w
YI12Yrj503xerE84e/yVkmqISiyp0hjFzPnzTgdsyRLA2Bk1dV2r9iK7b0nrXE7Z
3FC3T0P222hI2LJWajYyNArD38/doALMxHddMXpeWtWcBFVIMV0Y1Cy13CPT6JI
+yZ8sAWKH8npkP5LatDyD7Gux8JkLAg9FLDvkgvAURK8scopM+4AZYaN5aGyrzxkU
ZkD7VGyTeu7C8M1+4I2A1R8CsHtLAXaJ51iP+fcALaBbXt0yd6aR3hP3r6Ezo4kt
5wG+wyZ306cVpI4BLikXtWfL1x490fy/ofD7c15zTUKV5VAkn7qz2oHhbHjfr36r
6d/2G4c61A0bZoJJ65vFSECuALyg5P60ty1wAe4HYr0XX0GHQdR+tlmsrdZilez
kARtuzYsHxNMPLoBjwwH7NuXroSEx0T1E7srn3I3UIcudEvub56C1p33hjjXsjm
vUxuBL0Na2PFhAhZ54EXYt3xprhM/11kd4oRGokey/Y0RWt25woc5eqbRRr4Ww
EY9CrgSAdEK0C1dRuquqx9EraJhP7yZCVCdTqJ+csqEgvEwdh8XTeV2T5FpZToy/
Chs49sNXa6Vig8eBAa5Ge8UIviNacumcPKDm0Mc2Xtwqd1ReJa3L8QPSdwh1KGUw
Y/0f3xkfdBvhZDf4/D3zav3XC0ZX9V81Qh5LwGL0Hf38B1dejqXlBBA+837Gktj
Aqogt3zuUjKG66VMQ2Nfy3GXuFqmayDvuwnB2LKatFw7UX00fiPfJ9EavLgEr0
PJz1Un0HbVb+Gjm+mJhfd0dwULwHrn2Gx6u+UaJ4b2fRT00k45/IrnXG87WuSPy
oRn7F0u7yqyU7Q0fVNLpjjoTPV1zGPaX0e3F1tJvWRHFDJzCaoCkbh007P6qTh5
Lwb9YSM1cvDRFK7a9M6DzvH9DInVwcpwJMLaE2cXz7fjLzqrmVcrVwGM8IQELK/
QdQW0Fw5ySyBrDB98CxS2EfpuSHE6pt81+sJPZLmCb/N/WdmvNeyaxd09UQ
Ng+p04VSPKZogRzHeBUYUu5AEe7PYKMwtYE+geEy2x6yzJEQu8vTNU=
-----END ENCRYPTED PRIVATE KEY-----
blm4897a@u6-104-05:~/secu/rsa-keyfind-master$
```

FIGURE 4 – Création de la clé publique

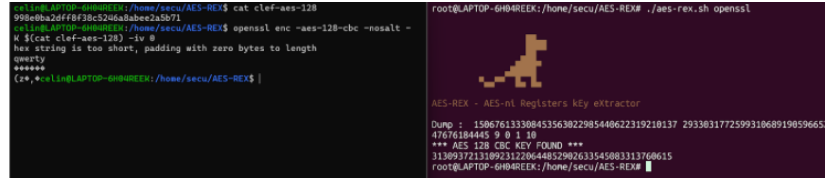
Partie 4 : AES-REX (Extraction avec les registres)

Dans cette partie, on utilise l'outil AES-REX qui va chercher les clés directement dans les registres du processeur (AES-NI).

La clé qu'on attendait était : 998e0ba2dff8f38c5246a8abee2a5b71. En modifiant le script pour afficher la variable \$dump, on a pu isoler les valeurs des registres XMM au moment du calcul :

— xmm0 = 150676133308453563022985440622319210137

— `xmm1` = 293303177259931068919059665247676184445
5375c097-f514-435e-941c-535815a4ff89



```
cell@LAPTOP-GH04REK:/home/secu/AES-REX$ cat clef-aes-128
998e8ba2dff8f38c5206a8abec2a5b71
cell@LAPTOP-GH04REK:/home/secu/AES-REX$ openssl enc -aes-128-cbc -nosalt -
K $(cat clef-aes-128) -iv 0
hex string is too short, padding with zero bytes to length
querty
aaaaaa
(2*,*cell@LAPTOP-GH04REK:/home/secu/AES-REX$ |

root@LAPTOP-GH04REK:/home/secu/AES-REX# ./aes-rex.sh openssl

AES-REX - AES-ni Registers Key extractor

Dump : 15067613330845356382298544062319210137 2933031772599310689190596652
47676184445 9 0 1 10
*** AES 128 CBC KEY FOUND ***
313093721310923120644852982633545883313708615
root@LAPTOP-GH04REK:/home/secu/AES-REX#
```

FIGURE 5 – Valeurs des registres xmm affichés par AES-REX

Analyse : La valeur hexadécimale de `xmm0` est 715B2AEEABA846528CF3F8DFA20B8E99. Comme les données sont stockées en *little endian*, il faut inverser les octets. Une fois l'inversion faite, on retombe exactement sur la clé attendue (998e...), ce qui confirme que l'extraction a fonctionné.